

```
import os
os.environ["KAGGLE_USERNAME"]="ayushgupta0250181"
os.environ["KAGGLE_KEY"]="3417b48bf2095674d83bbf8251e6610"
!kaggle datasets download -d JEHANBHATHENA/WEATHER-DATASET
```

Dataset URL: <https://www.kaggle.com/datasets/JEHANBHATHENA/WEATHER-DATASET>
License(s): CC0-1.0
Downloading WEATHER-DATASET.zip to /content
100% 587M/587M [00:03<00:00, 187MB/s]

```
!unzip /content/WEATHER-DATASET.zip
```

```
inflating: dataset/snow/1437.jpg
inflating: dataset/snow/1438.jpg
inflating: dataset/snow/1439.jpg
inflating: dataset/snow/1440.jpg
inflating: dataset/snow/1441.jpg
inflating: dataset/snow/1442.jpg
inflating: dataset/snow/1443.jpg
inflating: dataset/snow/1444.jpg
inflating: dataset/snow/1445.jpg
inflating: dataset/snow/1446.jpg
inflating: dataset/snow/1447.jpg
inflating: dataset/snow/1448.jpg
inflating: dataset/snow/1449.jpg
inflating: dataset/snow/1450.jpg
```

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Activation, Flatten, Conv2D
from tensorflow.keras.utils import image_dataset_from_directory, plot_model
import matplotlib.pyplot as plt
import numpy as np
```

```
#I've identified a critical missing import. The image_dataset_from_directory
```

```
DATASET_PATH = "dataset"
IMG_SIZE = (128,128)
BATCH_SIZE = 32
EPOCHS = 20
```

```
train_ds = image_dataset_from_directory(
    DATASET_PATH,
    validation_split=0.2,
    subset="training",
    seed=123,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)
```

Found 6862 files belonging to 11 classes.
Using 5490 files for training.

```
val_ds=image_dataset_from_directory(
    DATASET_PATH,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size = IMG_SIZE,
    batch_size = BATCH_SIZE
)
```

Found 6862 files belonging to 11 classes.
Using 1372 files for validation.

```
val_batches = tf.data.experimental.cardinality(val_ds)
test_ds = val_ds.take(val_batches // 2)
val_ds = val_ds.skip(val_batches // 2)
```

```
class_names = train_ds.class_names
print(class_names)
```

```
['dew', 'fogsmog', 'frost', 'glaze', 'hail', 'lightning', 'rain', 'rainbow',
```

```
plt.figure(figsize=(10,10))
for images,labels in train_ds.take(1):
    for i in range(9):
        plt.subplot(3,3,i+1)
        #before plt tensor flow ko change krna pdta h numpy me
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i]])
        plt.axis("off")
plt.show()
```

sandstorm



dew



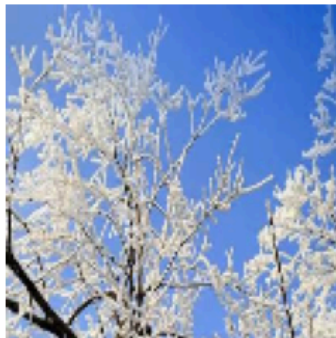
fogsmog



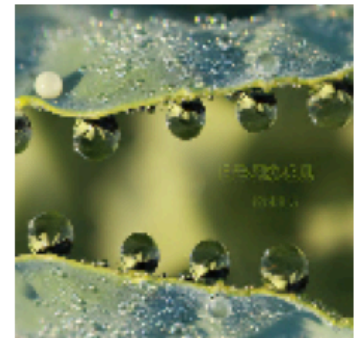
rain



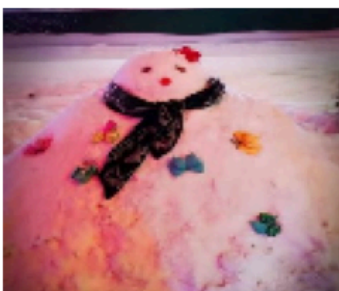
rime



dew



snow



sandstorm



rainbow



```
#class distribution
class_counts = {}
```

```
for class_name in os.listdir(DATASET_PATH):
    class_counts[class_name] = len(os.listdir(os.path.join(DATASET_PATH, class_name)))
print("Class Distribution :")
print(class_counts)
```

```
Class Distribution :
{'rainbow': 232, 'sandstorm': 692, 'frost': 475, 'dew': 698, 'fogsmog': 851,
```

```
import seaborn as sns

class_df = pd.DataFrame(class_counts.items(), columns=['Class', 'Count'])

class_df = class_df.sort_values(by='Count', ascending=False)

plt.figure(figsize=(12, 6))
sns.barplot(x='Class', y='Count', data=class_df, palette='viridis')
plt.title('Distribution of Images per Class')
plt.xlabel('Weather Class')
plt.ylabel('Number of Images')
plt.show()
```

/tmp/ipykernel_6210/1956784038.py:8: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in a future version. A default will be assigned.

```
sns.barplot(x='Class', y='Count', data=class_df, palette='viridis')
```



```
normalization_layer=Rescaling(1./255)
train_ds=train_ds.map(lambda x,y:(normalization_layer(x),y))
val_ds=val_ds.map(lambda x,y:(normalization_layer(x),y))
test_ds=test_ds.map(lambda x,y:(normalization_layer(x),y))
```

```
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.cache().prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.cache().prefetch(buffer_size=AUTOTUNE)
test_ds = test_ds.cache().prefetch(buffer_size=AUTOTUNE)
```

```
model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(IMG_SIZE[0], IMG_SIZE[1], 3),
    MaxPooling2D((2,2)),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D((2,2)),
    Conv2D(128, (3,3), activation='relu'),
    MaxPooling2D((2,2)),

    Flatten(),
    Dropout(0.5),
    Dense(128, activation='relu'),
    Dense(len(class_names), activation='softmax')
])
model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv2d.py:100:
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential_5"

Layer (type)	Output Shape	Param #
conv2d_15 (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d_15 (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_16 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_16 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_17 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_17 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten_5 (Flatten)	(None, 25088)	0
dropout (Dropout)	(None, 25088)	0
dense_10 (Dense)	(None, 128)	3,211,392
dense_11 (Dense)	(None, 11)	1,419

Total params: 3,306,059 (12.61 MB)

Trainable params: 3,306,059 (12.61 MB)

Non-trainable params: 0 (0.00 B)

Start coding or [generate](#) with AI.

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
```

```
metrics=['accuracy'])
```

```
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS
)
```

```

——— 24s 118ms/step - accuracy: 0.5122 - loss: 1.4560 - val_accuracy: 0.6171
——— 3s 17ms/step - accuracy: 0.6537 - loss: 1.0273 - val_accuracy: 0.6514 -
——— 3s 17ms/step - accuracy: 0.6927 - loss: 0.8967 - val_accuracy: 0.7029 -
——— 3s 17ms/step - accuracy: 0.7268 - loss: 0.8157 - val_accuracy: 0.7243 -
——— 3s 17ms/step - accuracy: 0.7656 - loss: 0.7005 - val_accuracy: 0.7114 -
——— 3s 17ms/step - accuracy: 0.7954 - loss: 0.6042 - val_accuracy: 0.7286 -
——— 3s 17ms/step - accuracy: 0.8078 - loss: 0.5567 - val_accuracy: 0.7071 -
——— 3s 17ms/step - accuracy: 0.8384 - loss: 0.4738 - val_accuracy: 0.7343 -
——— 3s 18ms/step - accuracy: 0.8545 - loss: 0.4317 - val_accuracy: 0.7014 -
——— 3s 17ms/step - accuracy: 0.8760 - loss: 0.3678 - val_accuracy: 0.7343 -
——— 3s 17ms/step - accuracy: 0.8965 - loss: 0.3093 - val_accuracy: 0.7143 -
——— 3s 17ms/step - accuracy: 0.9095 - loss: 0.2724 - val_accuracy: 0.7143 -
——— 3s 18ms/step - accuracy: 0.9209 - loss: 0.2334 - val_accuracy: 0.6900 -
——— 3s 17ms/step - accuracy: 0.9337 - loss: 0.1985 - val_accuracy: 0.7129 -
——— 3s 17ms/step - accuracy: 0.9330 - loss: 0.1901 - val_accuracy: 0.7114 -
——— 3s 17ms/step - accuracy: 0.9452 - loss: 0.1632 - val_accuracy: 0.6943 -
——— 3s 18ms/step - accuracy: 0.9443 - loss: 0.1736 - val_accuracy: 0.6957 -
——— 3s 17ms/step - accuracy: 0.9503 - loss: 0.1536 - val_accuracy: 0.7357 -
——— 3s 17ms/step - accuracy: 0.9468 - loss: 0.1449 - val_accuracy: 0.6886 -
——— 3s 17ms/step - accuracy: 0.9612 - loss: 0.1182 - val_accuracy: 0.7100 -

```

```
y_pred=model.evaluate(test_ds)
```

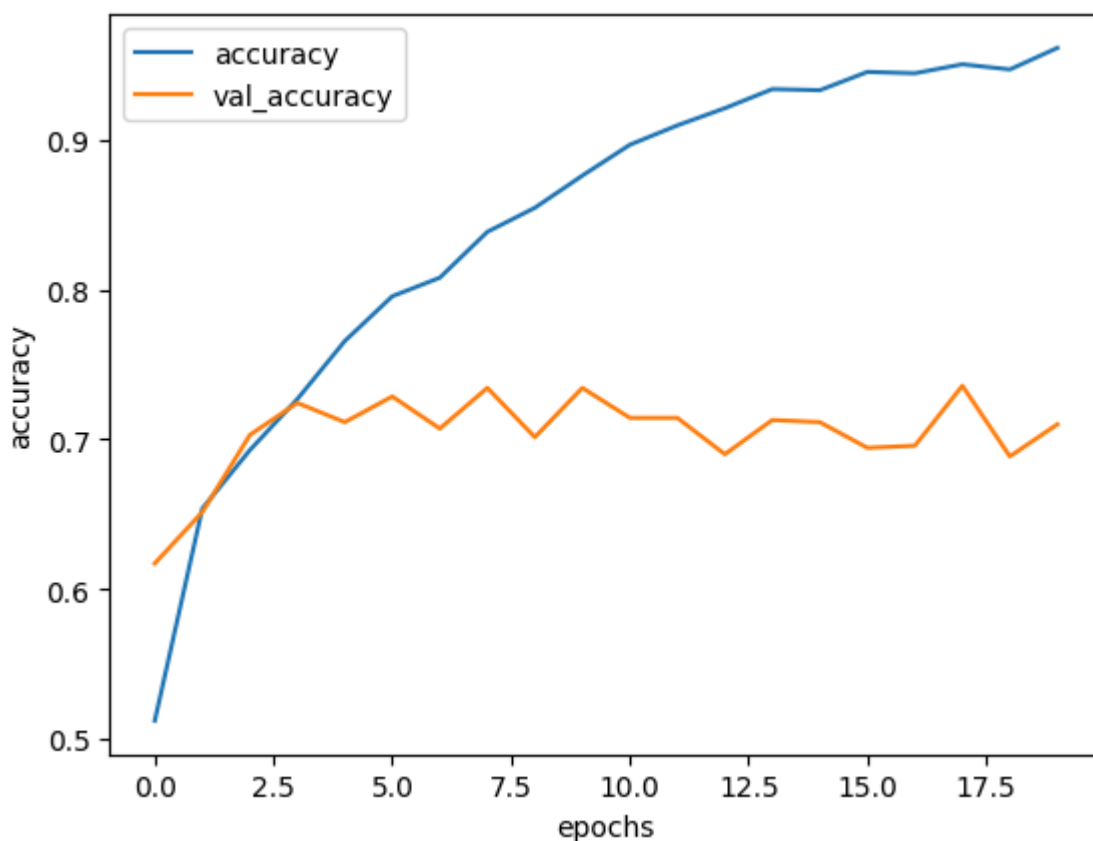
```
21/21 ————— 1s 58ms/step - accuracy: 0.6905 - loss: 1.6877
```

```

#plot accuracy graph
plt.plot(history.history["accuracy"],label="accuracy")
plt.plot(history.history["val_accuracy"],label="val_accuracy")
plt.xlabel("epochs")

```

```
plt.ylabel("accuracy")
plt.legend()
plt.show()
```



Start coding or [generate](#) with AI.

```
from sklearn.metrics import confusion_matrix

true_labels = []
predicted_labels = []

for images, labels in test_ds:
    for label in labels.numpy():
        true_labels.append(label)
    predictions = model.predict(images)
    for pred in np.argmax(predictions, axis=1):
        predicted_labels.append(pred)

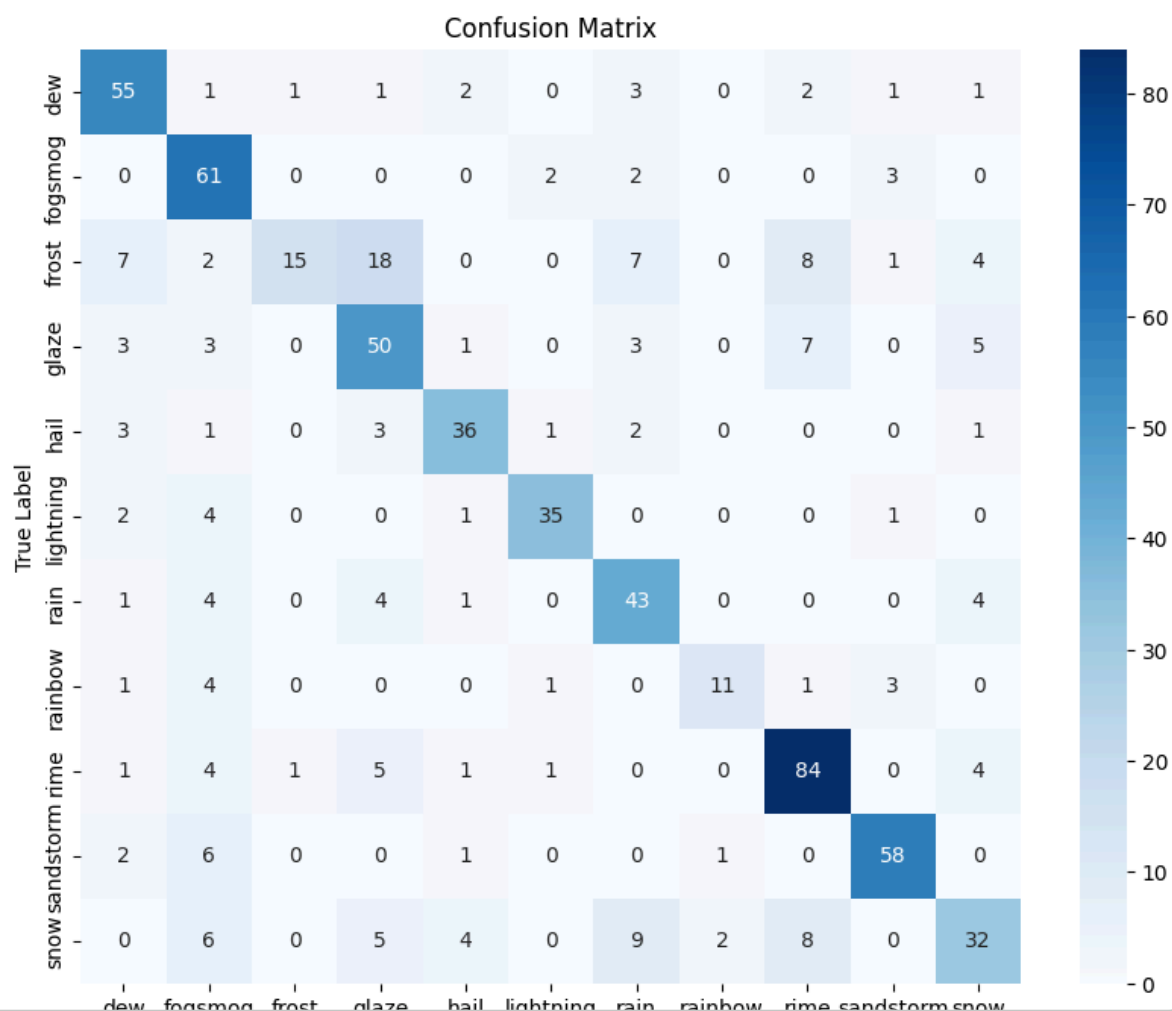
cm = confusion_matrix(true_labels, predicted_labels)

# Plot the confusion matrix
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=class_names,
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix')
plt.show()
```

```

1/1 ----- 1s 737ms/step
1/1 ----- 0s 89ms/step
1/1 ----- 0s 94ms/step
1/1 ----- 0s 96ms/step
1/1 ----- 0s 75ms/step
1/1 ----- 0s 77ms/step
1/1 ----- 0s 68ms/step
1/1 ----- 0s 82ms/step
1/1 ----- 0s 59ms/step
1/1 ----- 0s 44ms/step
1/1 ----- 0s 45ms/step
1/1 ----- 0s 43ms/step
1/1 ----- 0s 44ms/step
1/1 ----- 0s 42ms/step
1/1 ----- 0s 42ms/step
1/1 ----- 0s 47ms/step
1/1 ----- 0s 61ms/step
1/1 ----- 0s 45ms/step
1/1 ----- 0s 59ms/step
1/1 ----- 0s 44ms/step
1/1 ----- 0s 45ms/step

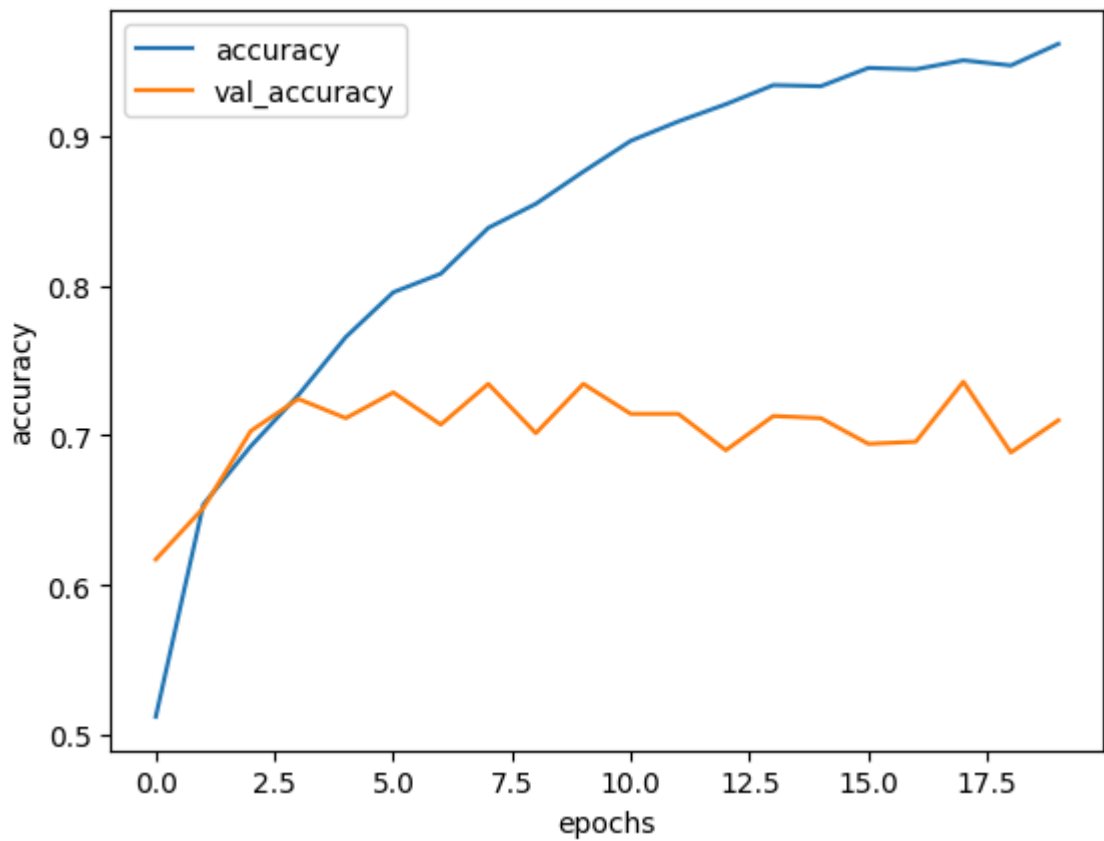
```



```

#plot accuracy graph
plt.plot(history.history["accuracy"],label="accuracy")
plt.plot(history.history["val_accuracy"],label="val_accuracy")
plt.xlabel("epochs")
plt.ylabel("accuracy")
plt.legend()
plt.show()

```

```
# Plot loss graph
plt.plot(history.history["loss"], label="loss")
plt.plot(history.history["val_loss"], label="val_loss")
plt.xlabel("epochs")
plt.ylabel("loss")
plt.legend()
plt.title("Loss vs. Epochs")
plt.show()
```

